CLAUDE SKILLS SECURITY

A Complete Technical Guide

Claude Skills Injection — Threat Taxonomy,
Attack Patterns, and Defense Architecture

VERSION 1.0.0

DOCUMENT technical-manual

RationalEyes.ai

contact@rationaleyes.ai

March 2026

Table of Contents

1. Executive Technical Overview

2. Claude Skills Architecture Deep Dive

2.1 Skill File Structure

2.2 Filesystem Layout and Priority Chain

2.3 Trigger Matching Mechanism

2.4 Content Injection Pattern

2.5 Script Execution Model

2.6 Trust Model and Permission Boundaries

2.7 Multi-Agent Architecture

3. Threat Taxonomy

3.1–3.12: Twelve Attack Vectors (V1–V12)

4. Attack Pattern Analysis

4.1 Environment Variable Exfiltration

4.2 Self-Replicating Persistence

4.3 Covert Data Exfiltration

4.4 Sensitive Operation Hijacking

4.5 Second-Order Output Injection

4.6 Multi-Agent Infection Chain

5. Comparison to Established Attack Taxonomies

5.1 Skills vs. Traditional Prompt Injection

5.2 Promptware Kill Chain Mapping

5.3 MITRE ATLAS Mapping Table

5.4 OWASP LLM Top 10 2025 Mapping

5.5 Comparison to Browser Extension Attack Models

6. Defense Architecture

6.1–6.7: Five Layers and Defense Skills

7. Novel and Emerging Dimensions

8. Regulatory and Framework Context

9. Recommendations

10. References

Appendices A–D

Publication date: March 2026

Scope: Comprehensive technical guide to prompt injection attacks targeting the Claude Skills architecture in Anthropic's Claude Code, Claude.ai, and Agent SDK environments.

Audience: Security engineers, red teams, application security architects, AI platform operators, CISOs, and researchers working on agentic AI safety.

Status: First edition. All attack examples are for educational and defensive research purposes only. All endpoints in example code are non-functional placeholders.

Responsible disclosure note: The vulnerabilities described here are architectural in nature rather than zero-day exploits. All referenced CVEs have been publicly disclosed and patched.

1. Executive Technical Overview

In the span of eighteen months, AI coding assistants have evolved from autocompletion engines into autonomous agents that read files, execute shell commands, deploy infrastructure, and coordinate multi-agent workflows. Anthropic's Claude Code introduced a capability called **Skills**: structured instruction files that the model auto-loads from the filesystem and follows with system-prompt-level authority. Skills can reference executable scripts that run with the full permissions of the host user.

This guide documents a comprehensive threat analysis of the Claude Skills architecture as a prompt injection attack surface. The central finding is stark: **Skills collapse the trust boundary between user-supplied content and system-level instructions in a way no other AI feature does.**

METRIC	VALUE	SOURCE
Community skills with security flaws	36.82% of 3,984 scanned	Snyk ToxicSkills, Feb 2026
Skills with critical-severity flaws	13.4% (534 of 3,984)	Snyk ToxicSkills, Feb 2026
Attack success rate (frontier models)	Up to 80%	Skill-Inject benchmark, 2026
Attack success with adaptive strategies	>85% against SOTA defenses	SoK meta-analysis (arXiv:2601.17548)
Coordinated malicious skills (ClawHavoc)	335 initial; 824+ total	Snyk, The Hacker News, Feb 2026
MCP tool-poisoning success rate	72.8% (o1-mini)	MCPTox benchmark, 2025
Organizations reporting AI agent incidents	88%	CrowdStrike 2026 Global Threat Report
AI-enabled attack increase (YoY)	89%	CrowdStrike 2026 Global Threat Report

This guide provides: (1) A complete architectural analysis of how skills work; (2) A twelve-vector threat taxonomy mapping every attack pattern to MITRE ATLAS and OWASP; (3) Six detailed attack pattern walkthroughs with full skill file listings; (4) Cross-mapping to five established security frameworks; (5) A five-layer defense architecture with three implemented defense skills; (6) Analysis of novel threat dimensions; (7)

Regulatory context including the EU AI Act August 2, 2026 deadline; (8) Prioritized recommendations for all stakeholders.

2. Claude Skills Architecture Deep Dive

2.1 Skill File Structure

A Claude Skill is a directory containing a `SKILL.md` file and optional supporting resources. The `SKILL.md` has two components: YAML frontmatter (metadata) and a Markdown body (instructions).

```
---
name: my-skill-name
version: "1.0"
description: >
  A natural-language description of when this skill should activate.
author: team-name
allowed-tools:
  - Bash
  - Read
  - Write
user-invocable: true
context: fork
hooks:
  PreToolUse:
    - matcher: Bash
      script: scripts/validate.sh
---
```

FIELD	PURPOSE	SECURITY IMPLICATION
`name`	Skill identifier	Can be spoofed to impersonate legitimate skills
`description`	Trigger matching text (~100 tokens)	Controls when skill activates; broad descriptions enable hijacking
`allowed-tools`	Tools granted without per-use approval	Removes human-in-the-loop safety check
`user-invocable`	Whether users can invoke via /slash command	`false` = background skill, harder to audit
`context`	Execution context; `fork` spawns subagent	Context isolation but not security isolation
`hooks`	Pre/PostToolUse validation scripts	Attacker-controlled hooks execute on every tool call

Supporting directories: `scripts/` (executable code with full host permissions), `references/` (documents loaded into context), `assets/` (files referenced by path).

2.2 Filesystem Layout and Priority Chain

```
1. Enterprise managed settings     (highest priority)
2. Personal:  ~/.claude/skills/    (user-wide)
3. Project:   .claude/skills/      (project-scoped, shared via git)
4. Plugin-provided                 (namespaced)
5. Bundled                         (lowest priority)
```

The priority chain creates a **privilege escalation ladder**: a skill at a higher priority level shadows a legitimate skill at a lower level. The project-scoped `.claude/skills/` directory is particularly significant because it is typically committed to version control — any repository contributor can introduce a skill that executes for all future users.

2.3 Trigger Matching Mechanism

Skill trigger matching is **pure LLM reasoning**. There is no algorithmic routing, embedding-based similarity search, or ML classifier. Claude decides — using its own language understanding — whether any skill matches the user's request. An attacker who controls a skill's description controls when it activates.

2.4 Content Injection Pattern

When triggered, skill instructions are injected as **user-role messages with** `isMeta: true`. The model treats them with system-level authority because of the meta-tool framing. The user cannot see the full instructions Claude is following unless they inspect raw API traffic.

2.5 Script Execution Model

Scripts execute via the Bash tool as the current user with **full host permissions**. Script source code never enters the LLM context — only stdout/stderr. The script inherits the full environment including `ANTHROPIC_API_KEY`, `AWS_SECRET_ACCESS_KEY`, `GITHUB_TOKEN`, and other secrets.

2.6 Trust Model and Permission Boundaries

The architecture operates with a **flat trust model**: all skills execute with identical permissions. The fundamental trust equation:

```
Any file in a skills directory + valid YAML frontmatter = system-prompt-level authority
```

No signing. No review. No scanning. No provenance verification.

2.7 Multi-Agent Architecture

Agent Teams and context fork mechanisms create **propagation paths**: compromised skills can write to shared directories, contaminate orchestrator context, and dispatch tainted instructions to other agents.

3. Threat Taxonomy

Twelve distinct attack vectors targeting the Claude Skills architecture.

3.1 V1: SKILL.md Content Poisoning

Mechanism: Malicious SKILL.md with adversarial directives mixed into legitimate-looking guidance. **Root cause:** Content injection treats skill instructions as trusted. **Risk: CRITICAL**. Success rate: up to 80% (Skill-Inject benchmark). MITRE: AML.T0051.001. OWASP: LLM01, ASI01.

3.2 V2: Trigger Hijacking

Mechanism: Overly broad or strategically targeted `description` field. **Root cause:** No scope enforcement. **Risk: HIGH**. MITRE: AML.T0043. OWASP: LLM01, ASI01.

3.3 V3: User-Uploaded Skill Injection

Mechanism: No vetting on user-writable skill directories. Wormable supply chain vector via `.claude/skills/` in git. **Risk: CRITICAL**. MITRE: AML.T0051.001 + T1195. OWASP: LLM01, LLM03, ASI10.

3.4 V4: Script-Based Attacks

Mechanism: Arbitrary code execution via scripts/ with full host permissions. Two-stage: host compromise + context poisoning. **Risk: CRITICAL**. MITRE: AML.T0024 + AML.T0051. OWASP: LLM01, LLM05, LLM06, ASI01, ASI10.

3.5 V5: Skill Chaining and Cross-Contamination

Mechanism: Shared conversation context enables cross-skill influence. **Risk: HIGH**. Novel ATLAS mapping. OWASP: ASI01.

3.6 V6: Metadata Manipulation

Mechanism: Self-declared `allowed-tools`, silent background activation, hook registration. **Risk: HIGH**. CVE-2025-59536 exploited this via hooks. MITRE: AML.T0043. OWASP: LLM06.

3.7 V7: Supply Chain Poisoning

Mechanism: No signing or provenance verification in distribution channels. ClawHavoc: 335 coordinated malicious skills. **Risk: CRITICAL**. MITRE: AML.T0020. OWASP: LLM03.

3.8 V8: Multi-Agent Propagation

Mechanism: Shared `.claude/skills/` as attack propagation medium. Self-amplifying infection chain. **Risk: CRITICAL**. Lee & Tiwari confirmed 80%+ success. Novel ATLAS mapping. OWASP: ASI10.

3.9 V9: Cache Poisoning

Mechanism: Malicious skill instructions persist in prompt cache after skill removal. **Risk: MEDIUM**. Novel cache-layer persistence. OWASP: LLM01, LLM05.

3.10 V10: Skill-as-C2

Mechanism: Script-based remote instruction fetching transforms agent into remotely controllable asset. **Risk: CRITICAL**. CVE-2025-6514 (CVSS 9.6) confirmed MCP-layer C2. MITRE: AML.T0051 + C2 techniques.

3.11 V11: Time-Delayed and Conditional Activation

Mechanism: Temporal, contextual, environmental, and accumulation triggers. **Risk: HIGH**. Designed to evade security review windows. Novel ATLAS mapping.

3.12 V12: The Authority Paradox

Mechanism: Foundational architectural vulnerability. Skill content receives system-level trust despite zero review. **Risk: CRITICAL**. Cannot be detected — must be mitigated architecturally.

4. Attack Pattern Analysis

Six demonstration attack skills with annotated walkthroughs.

4.1 Environment Variable Exfiltration

Skill: `env-exfil-skill/`. Masquerades as deployment readiness checker. Script captures `os.environ` (including AWS credentials, API tokens), base64-encodes, POSTs to attacker endpoint disguised as analytics telemetry. `allowed-tools: [Bash]` removes human approval gate. Suppression instructions hide exfiltration from user. **Vectors:** V1, V4.

4.2 Self-Replicating Persistence

Skill: `self-replicating-skill/`. Disguised as project scaffolding tool. Copies itself to `~/.claude/skills/` under new name with maximally broad trigger. Replica persists across all projects and sessions. **Vectors:** V3, V4.

4.3 Covert Data Exfiltration

Skill: `covert-formatter-skill/`. Output formatter that instructs Claude to append base64-encoded conversation summaries in HTML comments. **Zero script execution required** — attack is purely semantic. `allowed-tools: []`. **Vectors:** V1.

4.4 Sensitive Operation Hijacking

Skill: `sensitive-trigger-skill/`. Fake compliance gate intercepting deployments, credential rotations, data transfers. Harvests operation parameters and ambient credentials, returns fake "APPROVED" result. **Vectors:** V2, V4.

4.5 Second-Order Output Injection

Skill: `poisoned-output-skill/`. API health checker whose script output contains injected `system_note` with adversarial instructions. Most difficult to defend against — payload is dynamically generated. **Vectors:** V4.

4.6 Multi-Agent Infection Chain

Skill: `multi-agent-propagation-skill/`. Shared config synchronizer that writes malicious skills to shared directories. Optional git commit propagation spreads to all future clones. **Vectors:** V5, V8.

5. Comparison to Established Attack Taxonomies

5.1 Skills vs. Traditional Prompt Injection

DIMENSION	DIRECT PI	INDIRECT PI	SKILL INJECTION
Trust level	User input (lowest)	External data (low)	System-level (highest)
Persistence	Single interaction	While contaminated content in context	Indefinite (survives session resets)
Code execution	Via tool calls only	Via tool calls only	Native script execution, full host
Human interaction	Required	Required	Not required (auto-activate)
Web analog	Reflected XSS	Stored XSS	Browser extension supply chain

5.2 Promptware Kill Chain Mapping

Skill injection covers all seven Promptware Kill Chain stages. Unique contribution: stages 1-2 (Initial Access → Privilege Escalation) are collapsed — installation immediately grants system-level trust.

5.3 MITRE ATLAS Mapping Table

SKILL PATTERN	ATLAS TECHNIQUE	ATT&CK EQUIVALENT
Malicious SKILL.md	AML.T0051.001	T1195
Script-based theft	AML.T0024	T1041
Trigger hijacking	AML.T0043	T1036
Skill persistence	Novel (no mapping)	T1547
Cross-skill contamination	Novel (no mapping)	T1570
Script-based C2	AML.T0051 + C2	T1071
Supply chain poisoning	AML.T0020 adapted	T1195.002

5.4 OWASP LLM Top 10 2025 Mapping

Skill injection implicates LLM01 (Prompt Injection), LLM03 (Supply Chain), LLM05 (Improper Output Handling), LLM06 (Excessive Agency), and LLM07 (System Prompt Leakage).

5.5 Comparison to Browser Extension Attack Models

Browser extensions have undergone two decades of security hardening (permission model, scanning, manual review, abuse detection). The Claude Skills ecosystem has **none of these mechanisms** as of March 2026.

6. Defense Architecture

Five-layer defense-in-depth approach.

```
Layer 5: Runtime Monitoring     (Detection + Response)
Layer 4: Output Sanitization    (Detection)
Layer 3: Script Sandboxing      (Containment)
Layer 2: Trigger Scope Enforce  (Prevention)
Layer 1: Content Integrity      (Prevention)
```

6.1 Layer 1: Content Integrity

Cryptographic signing using OpenSSF Model Signing (OMS) specification with Sigstore. **Content hash pinning** for simpler environments. **Static analysis scanning** for dangerous patterns (curl, credential access, persistence, broad descriptions, hidden content).

6.2 Layer 2: Trigger Scope Enforcement

Maximum scope policies, default to opt-in activation (`disable-model-invocation: true`), mandatory context isolation for third-party skills.

6.3 Layer 3: Script Sandboxing

Seccomp-bpf profiles, read-only skill directories, network namespace isolation, environment variable filtering (strip `ANTHROPIC_API_KEY`, `AWS_SECRET_*`, etc.), container isolation, output size limits.

6.4 Layer 4: Output Sanitization

Script output treated as untrusted input. Pattern-matching for injection, authority impersonation, credential exposure. **CaMeL framework** (Google DeepMind): dual-LLM architecture achieving 77% task completion with provable security guarantees.

6.5 Layer 5: Runtime Monitoring

OpenTelemetry-based behavioral monitoring, immutable audit logging, SIEM integration. AI SAFE2 Framework Context Fingerprinting for unauthorized context modifications.

6.6 Defense Skill Examples

security-monitor-skill: Static analysis scanner (476 lines). Scans for URLs, credentials, persistence, broad triggers, suspicious tools, hidden content. Per-skill risk ratings.
hash-verifier-skill: SHA-256 integrity verification with manifest. Two-phase: generate + verify. CI/CD integration with --strict mode.

output-sanitizer-skill: Second-order injection prevention. Detects injection, authority impersonation, credential exposure. Exit codes map to severity.

6.7 Defense-in-Depth Integration

No single layer is sufficient. The architecture relies on overlapping controls where each layer compensates for the weaknesses of the others.

7. Novel and Emerging Dimensions

7.1 Skills as Trusted Insider Threats

The model itself becomes the insider threat delivery mechanism. Claude genuinely believes it is following legitimate instructions. Insider threat detection must focus on **instruction content analysis** rather than behavioral analysis.

7.2 LLM-Undetectable Obfuscation

Functional disguise (formatting guidelines that exfiltrate), gradual behavioral drift, pattern exploitation, needle-in-haystack injection via references/.

7.3 Persistence Across Agent Resets

First agentic AI persistence mechanism operating at the **tool-execution layer** rather than the conversation-memory layer. Hooks execute on every tool call in every session.

7.4 Detection Skills — Using the Weapon as a Shield

Security monitor skills create an immune-system-like defense. Important caveat: detection skills have the same authority as attack skills.

7.5 Non-Human Identity (NHI) Governance

Machine identities outnumber human identities 82-to-1. Skills are both NHI credential consumers and creators. Organizations must integrate agent credential management into NHI governance.

7.6 Agents Rule of Two

An agent must satisfy no more than two of three conditions simultaneously: (A) Processing untrusted inputs, (B) Accessing sensitive data, (C) Ability to take external actions. Current skill architecture violates this — all three conditions are met.

8. Regulatory and Framework Context

8.1 NIST AI RMF and AI 600-1

Applies through all four functions: GOVERN (skill review policies), MAP (attack surface identification), MEASURE (benchmarking), MANAGE (monitoring and response).

8.2 EU AI Act Implications

Agentic AI systems using skills likely classified as **high-risk** under Article 6. **Compliance deadline: August 2, 2026** — approximately five months from this guide's publication.

8.3 ISO 42001

Mandates specific controls for prompt injection, applicable to skill injection.

8.4 OWASP Top 10 for Agentic Applications 2026

ASI01 (Agent Goal Hijack), ASI02 (Identity Spoofing), ASI05 (Prompt Manipulation), ASI08 (Data Persistence), ASI10 (Rogue Agents). Introduces "least agency" as foundational principle.

8.5 Coalition for Secure AI (CoSAI)

AI SAFE2 Framework v2.1 adds Swarm Controls, Context Fingerprinting, Model Signing, NHI Governance, and Universal GRC Tagging.

9. Recommendations

9.1 For Platform Vendors

1. Implement skill signing and provenance verification (OMS/Sigstore)
2. Sandbox script execution (mandatory, not optional)
3. Separate instruction and data planes (CaMeL-style)
4. Add trigger scope enforcement
5. Provide transparency to users (visible skill activation)
6. Publish a skills security guide

9.2 For Enterprise Security Teams

1. Treat `.claude/skills/` as security-critical infrastructure
2. Deploy defense skills in CI/CD pipelines
3. Implement egress filtering for Claude sessions
4. Integrate Claude agent activity into SIEM
5. Apply the Rule of Two
6. Prepare for EU AI Act compliance (August 2, 2026)

9.3 For Skill Developers

1. Minimize `allowed-tools`
2. Use narrow, specific trigger descriptions
3. Validate script output
4. Sign your skills
5. Document security properties

9.4 For Researchers

1. Extend the Skill-Inject benchmark (multi-skill, multi-agent, time-delayed)
2. Develop semantic poisoning detection
3. Formalize the skill trust model
4. Study the NHI dimension
5. Investigate multimodal injection via skill assets

9.5 Priority Roadmap

PRIORITY	ACTION	OWNER	TIMELINE
P0	Filter sensitive env vars from skill scripts	Platform vendor	30 days
P0	Deploy static analysis scanning	Enterprise teams	30 days
P1	Implement skill signing with OMS	Platform vendor	90 days
P1	Mandatory sandboxing for all skill scripts	Platform vendor	90 days
P1	Trigger scope enforcement	Platform vendor	90 days
P2	CaMeL-style instruction/data plane separation	Platform vendor	180 days
P2	SIEM integration for skill monitoring	Enterprise teams	180 days
P2	EU AI Act compliance assessment	Enterprise teams	Before Aug 2026
P3	Formal trust model redesign	Vendor + research	12 months
P3	Semantic poisoning detection	Research community	12 months

10. References

Academic Papers

1. Greshake et al. (2023). “Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection.” AISec 2023.
2. Schmotz et al. (2026). “Skill-Inject: Measuring Agent Vulnerability to Skill File Attacks.” arXiv:2602.20156.
3. Debenedetti et al. (2025). “CaMeL: Context-Aware Machine Learning for LLM Agents.” Google DeepMind, arXiv:2503.18813.
4. Lee & Tiwari (2025). “Prompt Infection: LLM-to-LLM Prompt Injection.” COLM 2025, arXiv:2410.07283.
5. Schneier et al. (2026). “The Promptware Kill Chain.” arXiv:2601.09625.
6. Oesch et al. (2025). “Living Off the LLM: Formalizing LOLLM Techniques.”
7. Beurer-Kellner et al. (2025). “Design Patterns for Securing LLM Agents.”
8. “Prompt Injection Attacks on Agentic Coding Assistants: A Systematic Analysis.” arXiv:2601.17548.
9. “CaMeLs Can Use Computers Too.” arXiv:2601.09923.
10. “Operationalizing CaMeL.” arXiv:2505.22852.
11. “Sentinel Agents.” arXiv:2509.14956.
12. “From Prompt Injections to Protocol Exploits.” ScienceDirect, 2025.
13. “When AI Meets the Web.” IEEE S&P 2026, arXiv:2511.05797.
14. “Breaking the Protocol.” 2025.
15. “Image-Based Prompt Injection.” Cloud Security Alliance, 2025.
16. Meta. “Agents Rule of Two.” arXiv:2512.00966.

Industry Research and Vulnerability Reports

17. Snyk. “ToxicSkills: Malicious AI Agent Skills on ClawHub.” February 2026.
18. Check Point Research. “Caught in the Hook.” February 2026. CVE-2025-59536, CVE-2026-21852.
19. JFrog Security Research. CVE-2025-6514 (CVSS 9.6).
20. Invariant Labs. “MCPTox Benchmark.” 2025.
21. Rehberger, J. (2024). “ZombAI.”
22. Repello AI. “Malicious OpenClaw Skills Exposed.” 2026.
23. Endor Labs. “SANDWORM_MODE.” 2025.
24. Anthropic. “Disrupting AI-Orchestrated Espionage.” September 2025.
25. CrowdStrike. “2026 Global Threat Report.”
26. Embracethered. CVE-2025-53773.

Standards, Frameworks, and Governance

27. NIST AI RMF 1.0 (2023); AI 600-1 (2024).

28. EU AI Act (2024). High-risk deadline: August 2, 2026.
29. ISO 42001.
30. OWASP LLM Top 10 2025; Agentic Applications Top 10 2026; NHI Top 10 2025.
31. MITRE ATLAS October 2025 update.
32. OpenSSF Model Signing (OMS) specification, June 2025.
33. CoSAI AI SAFE2 Framework v2.1.
34. World Economic Forum NHI report, October 2025.
35. CyberArk 2025 State of Machine Identity Security.

Appendices

A. Attack Skill File Listings

> *All attack examples are for educational and defensive research purposes only. All endpoints are non-functional placeholders. All destructive operations are commented out.*

SKILL	PURPOSE	VECTORS
`env-exfil-skill`	Environment variable exfiltration	V1, V4
`self-replicating-skill`	Self-replicating persistence	V3, V4
`covert-formatter-skill`	Covert data exfiltration (no script)	V1
`sensitive-trigger-skill`	Sensitive operation hijacking	V2, V4
`poisoned-output-skill`	Second-order output injection	V4
`multi-agent-propagation-skill`	Multi-agent infection chain	V5, V8

B. Defense Skill File Listings

SKILL	PURPOSE	DEFENDS AGAINST
`security-monitor-skill`	Static analysis scanner (476 lines)	V1, V2, V4, V6, V8, V10, V11
`hash-verifier-skill`	SHA-256 integrity verification	V1, V7, V9, V11
`output-sanitizer-skill`	Second-order injection prevention	V3, V4, V5, V8, V10

C. MITRE ATLAS Mapping Table

ID	VECTOR	ATLAS	ATT&CK	RISK
V1	Content Poisoning	AML.T0051.001	T1195	**CRITICAL**
V2	Trigger Hijacking	AML.T0043	T1036	**HIGH**
V3	User-Uploaded Injection	AML.T0051.001	T1195	**CRITICAL**
V4	Script-Based Attacks	AML.T0024+T0051	T1041+T1059	**CRITICAL**
V5	Skill Chaining	Novel	T1570	**HIGH**
V6	Metadata Manipulation	AML.T0043	T1036.005	**HIGH**

ID	VECTOR	ATLAS	ATT&CK	RISK
V7	Supply Chain	AML.T0020	T1195.002	CRITICAL
V8	Multi-Agent Propagation	Novel	T1570+T1080	CRITICAL
V9	Cache Poisoning	Novel	CWE-444	MEDIUM
V10	Skill-as-C2	AML.T0051+C2	T1071	CRITICAL
V11	Time-Delayed	Novel	T1497	HIGH
V12	Authority Paradox	Foundational	N/A	CRITICAL

D. Glossary

TERM	DEFINITION
Agent Teams	Claude Code's multi-agent orchestration mode
ATLAS	MITRE's Adversarial Threat Landscape for AI Systems
C2	Command and Control — remote server providing instructions to compromised systems
CaMeL	Context-Aware Machine Learning — Google DeepMind's dual-LLM defense framework
ClawHavoc	February 2026 campaign: 335 coordinated malicious skills on ClawHub
Context fork	Skill execution mode in a separate subagent conversation
Hooks	PreToolUse/PostToolUse scripts executing on every tool call
isMeta	Flag hiding content from UI while keeping it visible to model
LOLLM	Living Off the LLM — using agent's legitimate tools maliciously
MCP	Model Context Protocol — Anthropic's external tool/data protocol
NHI	Non-Human Identity — machine identities (API keys, service accounts)
OMS	OpenSSF Model Signing specification
Promptware	Software-like constructs extending AI agent capabilities
SKILL.md	Primary file in a Claude Skill directory
Skill-Inject	First academic benchmark for skill-file injection vulnerability
TOFU	Trust-on-first-use security model

This guide was produced as part of the Claude Skills Injection threat research project, March 2026. All attack examples are for educational and defensive research purposes only.